PYTHON PROGRAMMING MINI PROJECT

Personal To-Do List Application

Internship Project Report

Student Name: **Kanak Chaudhary**

Internship Program: **VaultofCodes Python Internship**

Assignment: **Assignment 4**

Programming Language: **Python 3**

Development Type: **Command-Line Application (CLI)**

Submitted as part of the VaultofCodes Python Programming Internship 2026

1. Project Objective

The objective of this project is to build a Personal To-Do List Application that helps users manage their daily tasks in an organized and efficient way. The application allows users to add new tasks, edit existing tasks, mark tasks as completed, delete tasks, and organize tasks into categories such as Work, Study, or Personal.

To make sure that no task information is lost between program runs, all tasks are stored in a JSON file. Every time the program starts, it automatically loads the previously saved tasks, and every time a change is made, the updated task list is saved back to the file. This gives the user a simple but reliable command-line tool for daily task management.

2. Problem Statement

In everyday life, people often have to remember several tasks at once, such as college assignments, internship deadlines, household chores, or personal goals. When these tasks are not written down or tracked properly, it becomes easy to forget them or lose track of what has already been completed.

A digital to-do list solves this problem by giving the user a single, organized place to record tasks, track their progress, and review what is pending. Compared to a paper list, a program-based to-do list is faster to update, easier to search, and can automatically remember the task history through file storage. This project was built to demonstrate how a simple Python program can solve this everyday productivity problem using core programming concepts.

3. Technologies Used

The following technologies and concepts were used to build this application:

- **Python 3 –** the core programming language used to write the entire application.
- **Object-Oriented Programming (OOP) –** used to represent each task as an object with its own properties, such as title, category, and completion status.
- **JSON File Handling –** used to permanently store task data in a tasks.json file so tasks are not lost when the program closes.
- **Lists and Dictionaries –** used internally to store, search, and manage collections of task objects.
- **Functions and Modular Programming –** the program logic is divided into small, reusable functions, which makes the code easier to read, test, and maintain.
- **Command-Line Interface (CLI) –** the user interacts with the application entirely through a textbased menu in the terminal.

4. Project Features

The application provides the following features to the user:

- **Add Task –** allows the user to create a new task with a title and category.
- **View All Tasks –** displays the complete list of tasks, both completed and pending.
- **View Completed Tasks –** filters and shows only the tasks that have been marked as done.
- **View Pending Tasks –** filters and shows only the tasks that are still incomplete.
- **View Tasks by Category –** lets the user see tasks grouped under a specific category, such as Study or Work.
- **Edit Task –** allows the user to update the details of an existing task.
- **Mark Task as Completed –** changes the status of a task once it has been finished.
- **Delete Task –** permanently removes a task from the list.
- **JSON Data Persistence –** saves all tasks to a JSON file so the data is not lost after the program closes.
- **Automatic Loading of Saved Tasks –** reads the JSON file at startup so previous tasks are available immediately.
- **Input Validation –** checks user input to prevent invalid menu choices or empty task entries.
- **Menu-Driven Interface –** presents all available options in a clear, numbered menu for easy navigation.

5. Project Structure

The project files are organized in the following structure:

```
Assignment-4
└── Task-1_Personal-ToDo-List
    ├── todo.py
    ├── tasks.json
    └── README.md
```

- **todo.py –** contains the complete source code of the application.
- **tasks.json –** stores all task data so it persists between program runs.
- **README.md –** provides a short description and usage instructions for the project.

6. Implementation Overview

The application is built around a small number of well-defined components, each responsible for one part of the program's behaviour. The table below summarizes the purpose of each major component.

`Task class`	Represents a single task as an object, storing details such as the title, category, and completion status. Each Task object can be converted into a dictionary for saving, and rebuilt from a dictionary when loading.
`save_tasks()`	Converts the current list of Task objects into dictionaries and writes them to tasks.json, so all changes are stored permanently.
`load_tasks()`	Reads tasks.json when the program starts and rebuilds the list of Task objects from the saved data, so previous tasks are available immediately.
`add_task()`	Collects task details from the user, creates a new Task object, and adds it to the task list.
`view_tasks()`	Displays tasks to the user, with support for viewing all tasks, only completed tasks, only pending tasks, or tasks in a chosen category.
`mark_task_completed()`	Updates the status of a selected task to completed.
`edit_task()`	Allows the user to select an existing task and update its details, such as title or category.
`delete_task()`	Removes a selected task permanently from the task list.
`print_task_list()`	Formats and prints the task list in a clean, numbered, and readable layout in the terminal.
`main()`	Runs the program loop: loads saved tasks, displays the menu, calls the correct function based on user choice, and saves tasks after every change.

In short, **each task is handled as an object**, which is converted into a dictionary before being saved and reconstructed back into an object when the data is loaded. This design keeps the task data organized in memory while still allowing it to be stored in a simple, human-readable `tasks.json` file.

7. Program Flow

The overall flow of the application follows a simple and predictable sequence, as described below:

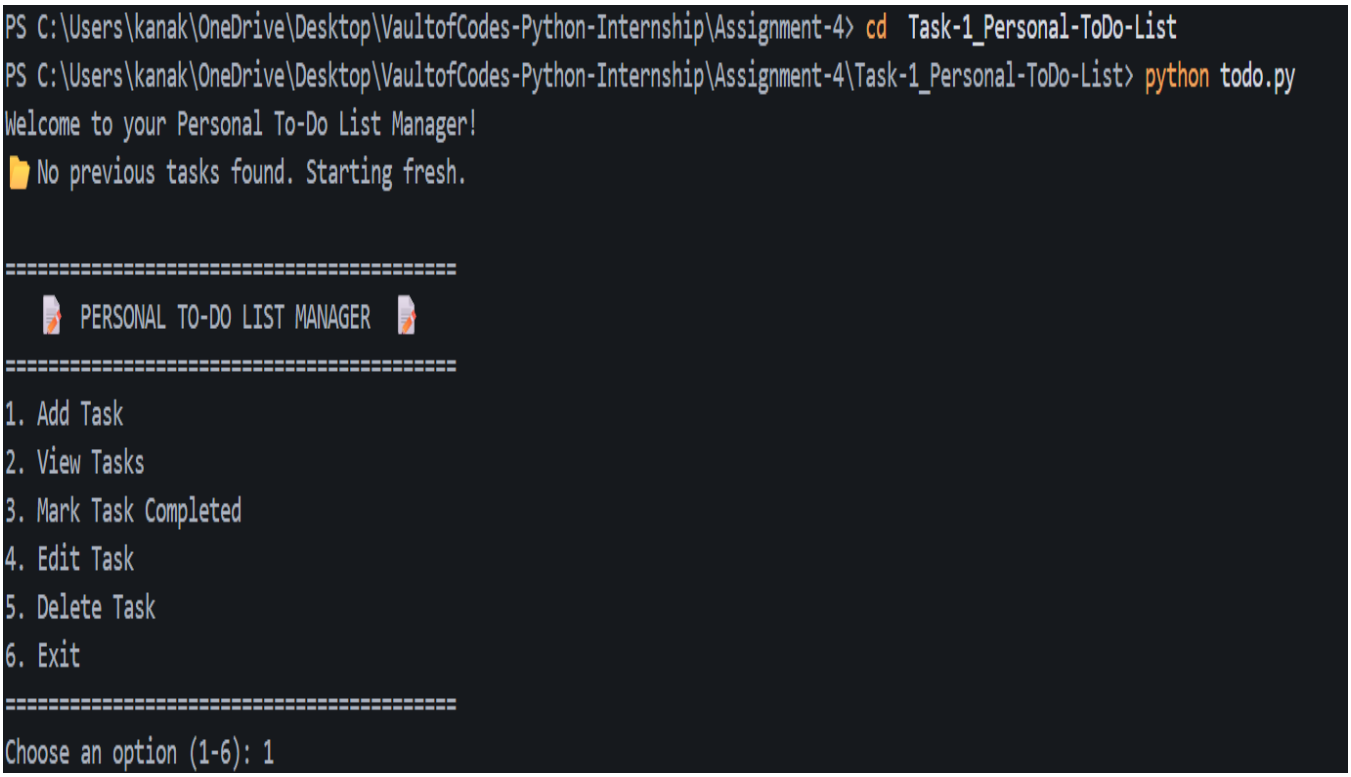
1	Start Application – the program begins execution when the user runs todo.py.
2	Load Existing Tasks – the program reads tasks.json and loads any previously saved tasks into memory.

3	Display Main Menu – the user is shown a numbered list of available options, such as Add Task, View Tasks, Edit Task, and Delete Task.
4	Perform User Operations – based on the user's choice, the corresponding function is called to add, view, edit, complete, or delete a task.
5	Save Updated Tasks – after every change, the updated task list is written back to tasks.json so no data is lost.
6	Exit Program – when the user chooses to quit, the program closes safely, with all changes already saved.

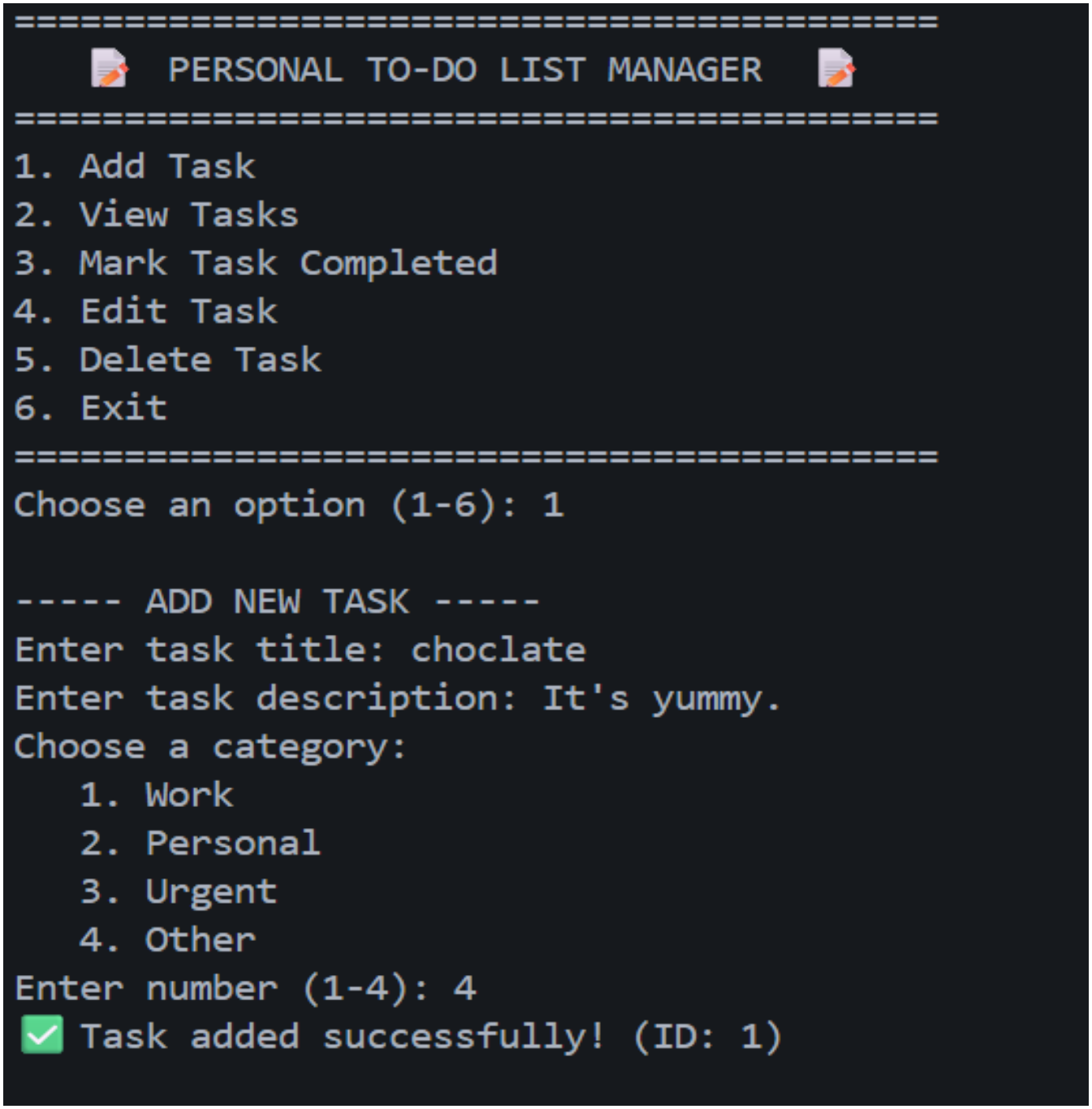
8. Output Screenshots

This section is reserved for screenshots of the application in action. Screenshots should be inserted in place of the placeholders below before final submission.

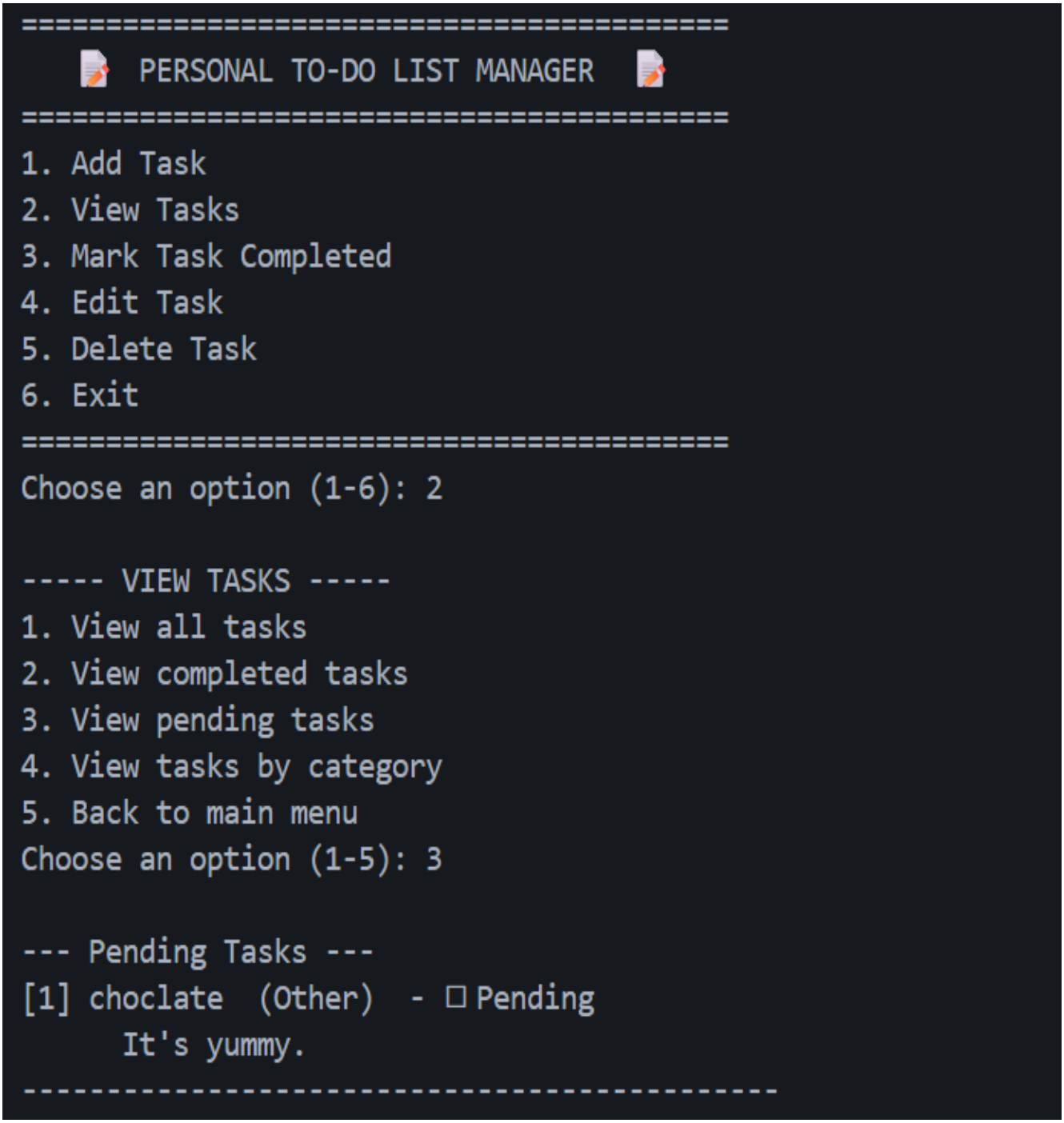
Screenshot 1 – Main Menu



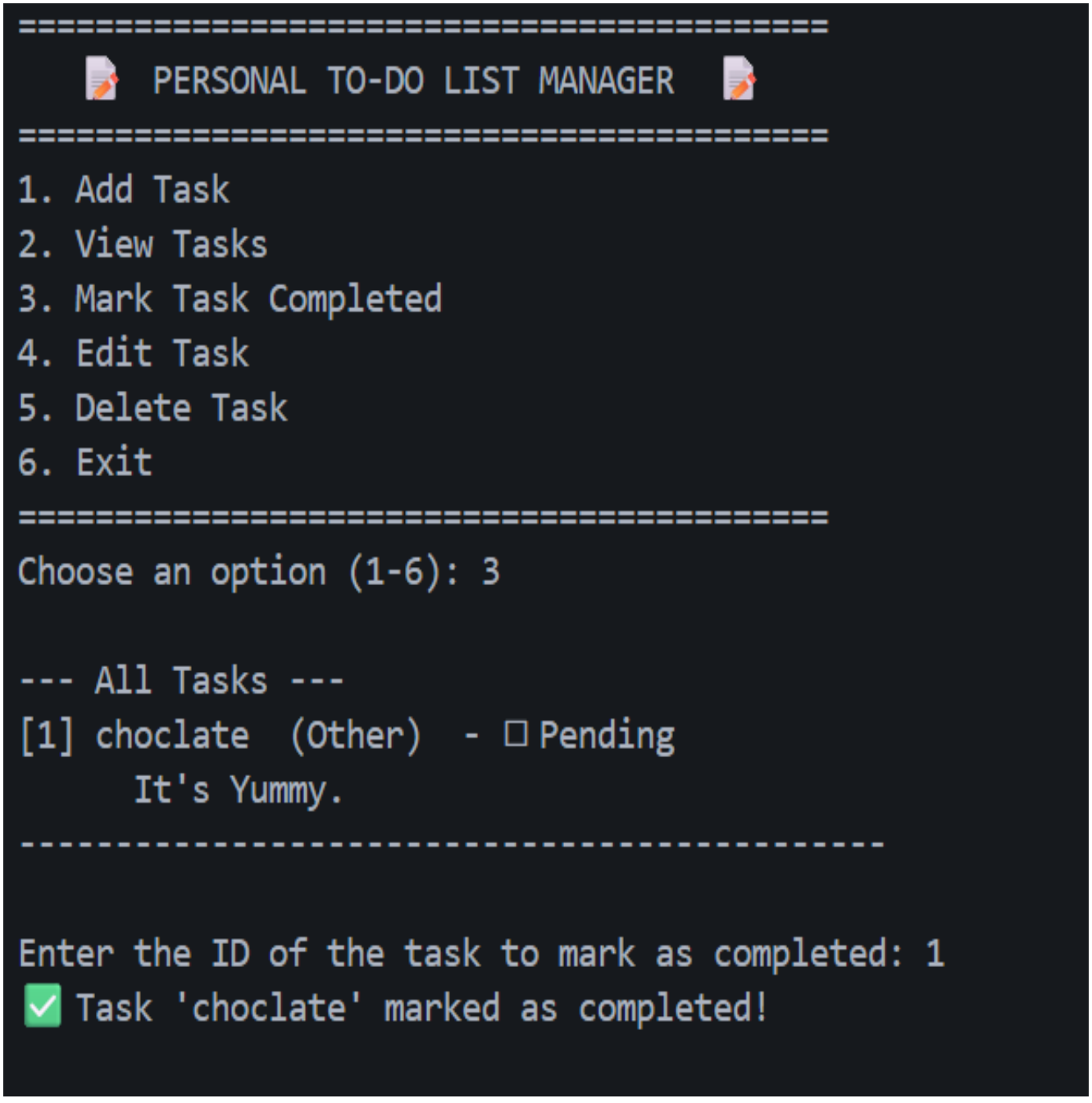
Screenshot 2 – Add Task



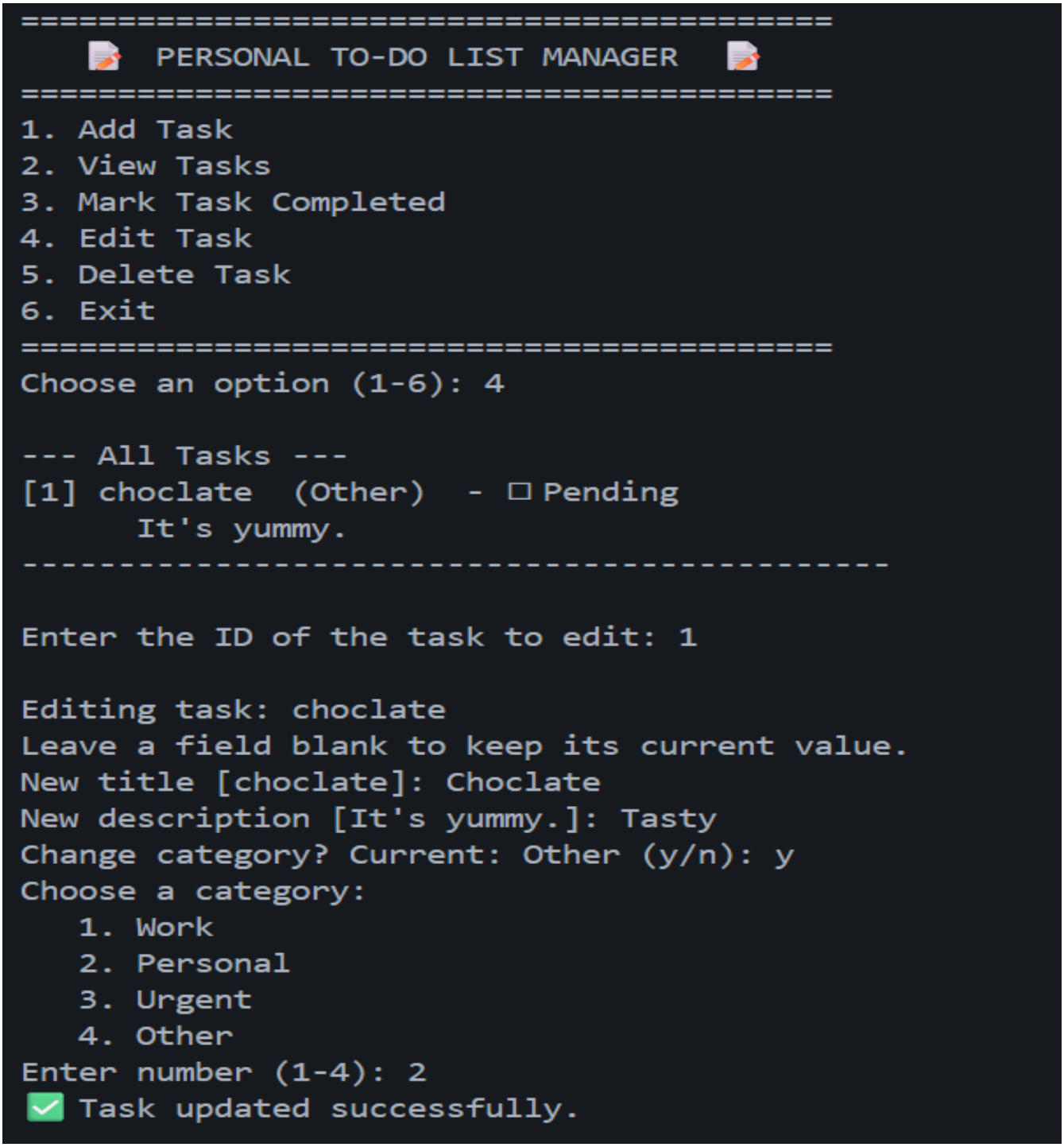
Screenshot 3 – View Tasks



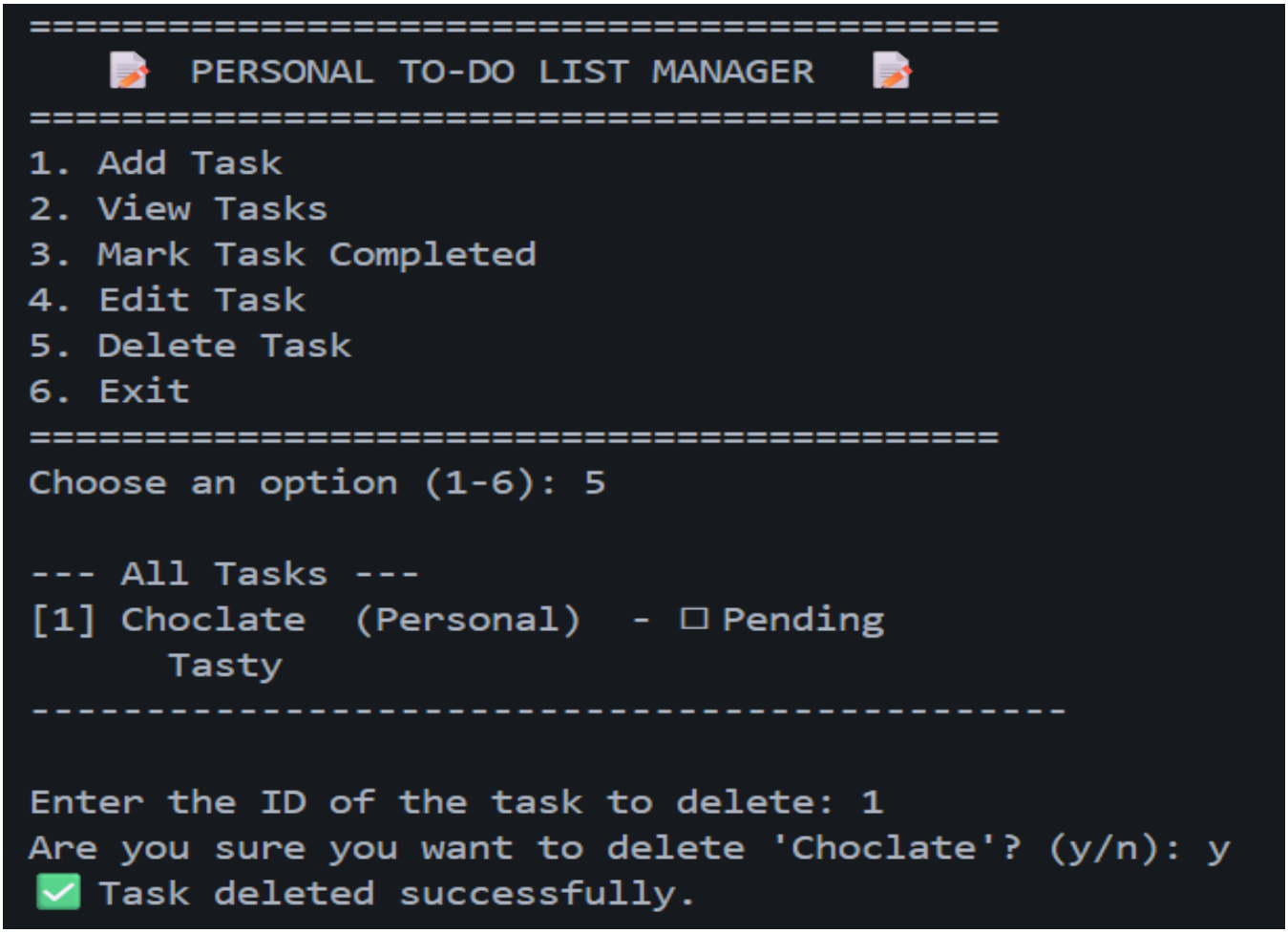
Screenshot 4 – Mark Task Completed



Screenshot 5 – Edit Task



Screenshot 6 – Delete Task



9. Learning Outcomes

Working on this project helped in strengthening several important programming concepts:

- **Object-Oriented Programming –** learned how to design a Task class and use objects to represent real-world entities in a program.
- **JSON File Handling –** learned how to read from and write to JSON files to store structured data permanently.

- **Functions –** learned how to break a program into small, reusable functions, which made the code cleaner and easier to debug.
- **Input Validation –** learned how to check and handle user input so the program does not crash on invalid entries.
- **CLI Application Development –** learned how to design a clear, menu-driven interface that is easy for a user to navigate in the terminal.
- **Data Persistence –** understood how to make an application "remember" data across multiple runs by saving state to a file.

10. Future Scope

The current version of the application covers the core requirements of a to-do list manager, but there are several ways it could be extended in the future:

- **Task Priority –** adding priority levels such as High, Medium, and Low to help users focus on urgent tasks first.
- **Due Dates –** allowing users to set deadlines for tasks and get a view of upcoming or overdue items.
- **Search Tasks –** adding a search feature to quickly find a specific task by keyword.
- **GUI using Tkinter –** building a graphical user interface so the application is easier to use for nontechnical users.
- **Database Integration –** replacing the JSON file with a database such as SQLite for larger and more reliable data storage.
- **Notifications and Reminders –** adding alerts to remind users of pending or upcoming tasks.

11. Conclusion

The Personal To-Do List Application successfully meets the goal of providing a simple and practical tool for managing daily tasks through the command line. Through this project, core Python concepts such as object-oriented programming, file handling, and modular function design were applied to build a working, real-world application.

The use of JSON for data persistence ensured that user tasks remain safe and available across multiple sessions, while the menu-driven CLI made the application easy to use despite having no graphical interface. Overall, this assignment demonstrates a solid understanding of Python fundamentals and provides a strong foundation that can be extended with more advanced features in the future.